Digital Audio Signal Processing and Synthesis

UJM 2026-2027 - DIGICREA(1), CCNT(3)

Pierre Cochard, INSA Lyon, Inria, Emeraude

2026-10-20

Hello!

- My name is **Pierre**, nice to meet you all!
- I'm currently working as a **research engineer** in Inria/INSA team **Emeraude**, working among other things on a PhD Thesis intitled *Compilation and Optimization methods for Digital Audio Signal Processing and Synthesis on hardware architectures*.
- I'm mainly a **software engineer**, a bit specialized in **audio** and **compilation**, I'm involved in the development/maintenance of tools such as:
  - **Syfala** (Faust-to-FPGA compilation toolchain)
  - **FloPoCo/HAriCo** (Hardware Arithmetic Core Generator)
  - **Emeraude-MLIR** (Compilation framework)
  - **Faust** (but not really…)

Learning Audio DSP & synthesis stuff

- **Sound** (physics)
- **Signal** & **transduction**:
  - **Analog** (electrical) **signal**
  - **Digital** Audio signal
- Audio DSP with the **Faust programming language**
  - Playground & Basics
  - Filtering & Synthesis
  - Time, Buffers & Delays
  - Controls (GUI, MIDI, OSC, …)
  - Exporting to different targets

- Audio DSP with **C++**:
  - Creating a **VCV-Rack Module**

Agenda

October

- Tuesday **20th** 14h-18h (**4 hours**)

November

- Tuesday **3rd**: 9h-12h (**3 hours**)
- Thursday **5th**: 13h-16h
- Tuesday **10th**: 9h-12h
- Thursday **12th**: 13h-16h
- Tuesday **17th**: 9h-12h
- Thursday **19th**: 13h-16h
- Tuesday **24th**: 9h-12h
- Thursday **26th**: 13h-16h

December

- Tuesday **1st**: 9h-12h
- Thursday **3rd**: 13h-16h

Evaluation

- Individual & group **projects** (deadline TBD)

Resources:

https://pchdev.github.io/ujm-dsp

Sound & Signal ……

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.
- Sound has the following properties:
  - **Speed**: ~343m/s in air

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.
- Sound has the following properties:
  - **Speed**: ~343m/s in air
  - **Amplitude**: measured in Pascals (Pa)

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.
- Sound has the following properties:
  - **Speed**: ~343m/s in air
  - **Amplitude**: measured in Pascals (Pa)

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.

- Sound has the following properties:
  - **Speed**: ~343m/s in air
  - **Amplitude**: measured in Pascals (Pa)
  - **Period**: time between two oscillations
  - **Wavelength**: distance between two oscillations

Sound (physical properties)

- Sound is a **pressure wave**, that propagates through a **medium** (gas, liquid, or solid).
- **Propagation** is caused by the **oscillation** (*vibration*) of the medium's **particles**, around their *equilibrium* positions.

- Sound has the following properties:
  - **Speed**: ~343m/s in air
  - **Amplitude**: measured in Pascals (Pa)
  - **Period**: time between two oscillations
  - **Wavelength**: distance between two oscillations

  - **Frequency**: cycles per second
    - in Hertz (Hz, kHz, MHz)
  - **Spectrum** (or timbre)

Sound perception

- Our perception of sound is made from the conversion of the **vibrations** reaching our **eardrums** to a **signal of nerve impulses**, transmitted and interpreted by the brain.
- **Human ears** can typically identify sounds ranging from **20 Hz to 20 kHz**.
- Other animal hearing ranges (a little *quizz*):

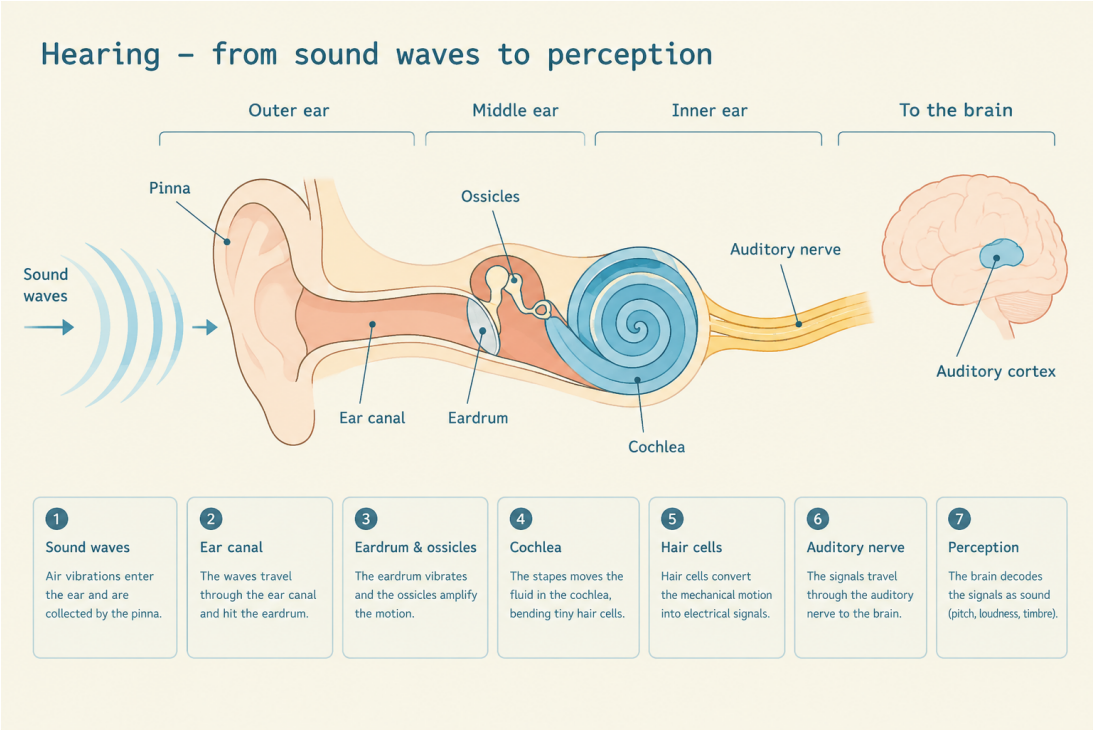
- **Bat**:
  - 2 kHz to 110 kHz,

- **Porpoise**:
  - 75 Hz to 150 kHz,

- **Cat**:

  - 45 Hz to 64 kHz,
- **Elephant**:
  - 5 Hz to 12 kHz
- **Chicken**:
  - 125 Hz to 2 kHz,
- **Owl**:
  - 100 Hz to 18 kHz,

Sound perception

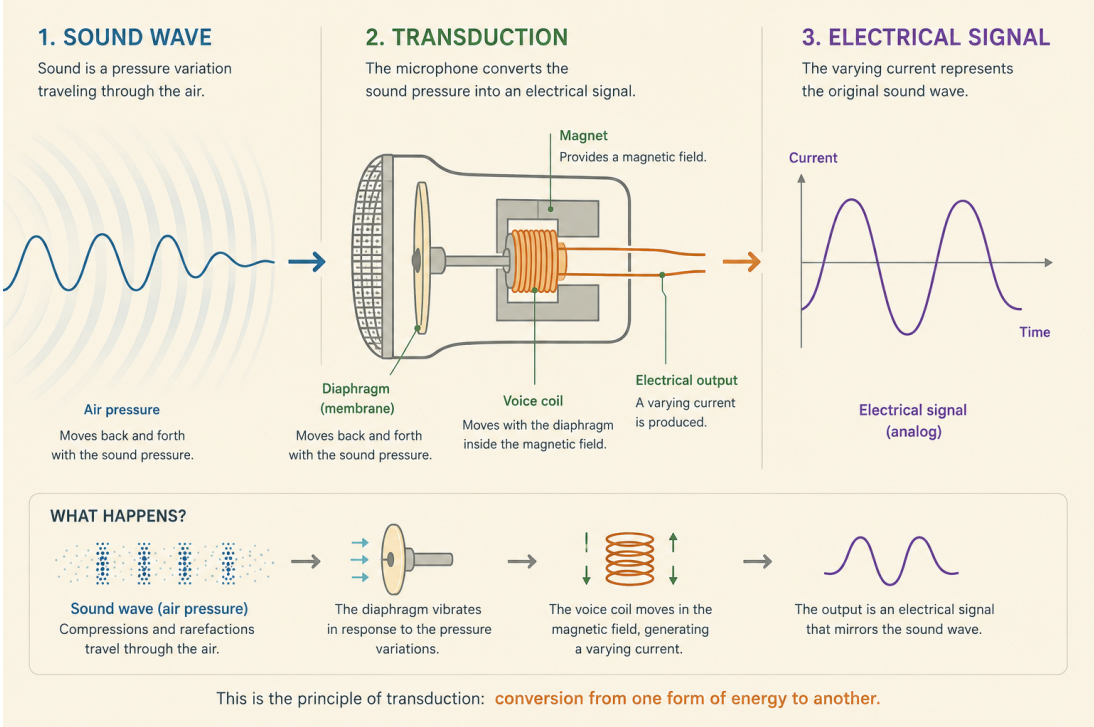


Signal

- A signal describes the **evolution** of a certain type of *data* **over time**, like the vibration of an entity: the **membrane of a microphone**.
- Just like sound waves turning into nerve impulses, the analyzed data usually needs to be first **converted to another physical unit**, or domain (***transduction***), in order to adapt to measurement/processing tools.

- The vibration of a microphone's membrane is, for instance, usually converted to **continuous electrical current** before it can be processed and/or analyzed. In this case, the signal is said to be ***analog***.

Signal (transduction)



Signal

- With an **oscilloscope**, we can measure the **amplitude of a signal** at a given fixed point in time (***time-domain***), through the visualisation of its ***waveform***.

Signal

- With an **oscilloscope**, we can measure the **amplitude of a signal** at a given fixed point in time (***time-domain***), through the visualisation of its ***waveform***.

Signal

- With an **oscilloscope**, we can measure the **amplitude of a signal** at a given fixed point in time (***time-domain***), through the visualisation of its ***waveform***.
- An analog signal can already be processed as it is with **analog effects**: *tape delay*, *distortion*, *chorus*, *reverberation* (spring, plate), etc.

Signal

- With an **oscilloscope**, we can measure the **amplitude of a signal** at a given fixed point in time (***time-domain***), through the visualisation of its ***waveform***.
- An analog signal can already be processed as it is with **analog effects**: *tape delay*, *distortion*, *chorus*, *reverberation* (spring, plate), etc.
- On the other hand, it is difficult to extract precise information about **frequency and spectrum**. For this purpose, it's far more efficient to switch to the ***frequency domain***, which requires the analog signal to be **turned into a digital signal**…

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves*** of different **phase**, **amplitude**, and **frequency**.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves*** of different **phase**, **amplitude**, and **frequency**.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.
- The "*waveform*" representation (**time-domain**) doesn't really help us when analyzing complex, **aperiodic** sounds.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.
- The "*waveform*" representation (**time-domain**) doesn't really help us when analyzing complex, **aperiodic** sounds.
- While it's possible to analyze the *spectral contents* of an **analog** signal with **filter banks**, the resulting precision is rather *low*, and unpractical to use.

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.
- The "*waveform*" representation (**time-domain**) doesn't really help us when analyzing complex, **aperiodic** sounds.
- While it's possible to analyze the spectral contents of an analog signal with filter banks, the resulting precision is rather low, and unpractical to use.
- **Frequency domain** on other hand allows us to decompose much more precisely a given sound into its *frequential components*, with a technique called ***Fast-Fourier-Transform*** (**FFT**).

Signal: Spectrum

- Any sound can be decomposed into a ***sum of sine waves***.
- A **sawtooth wave**, for instance, is composed of a **fundamental frequency** and a certain number of **harmonics**, which are ***multiples of the fundamental***.
- The "*waveform*" representation (**time-domain**) doesn't really help us when analyzing complex, **aperiodic** sounds.
- While it's possible to analyze the spectral contents of an analog signal with filter banks, the resulting precision is rather low, and unpractical to use.

- **Frequency domain** on other hand allows us to decompose much more precisely a given sound into its *frequential components*, with a technique called ***Fast-Fourier-Transform*** (**FFT**).

Digital Signal

- To *digitize* a continuous signal implies ***discretizing it***. This is made possible by an **Analog-to-Digital Conversion** (**ADC**) process, which implies two key elements: ***sampling*** and ***quantization***.

Digital Signal

- To *digitize* a continuous signal implies ***discretizing it***. This is made possible by an **Analog-to-Digital Conversion** (**ADC**) process, which implies two key elements: ***sampling*** and ***quantization***.

- **Sampling** means taking a sample of a signal at a certain frequency/rate (***sample rate***).

Digital Signal

- To *digitize* a continuous signal implies ***discretizing it***. This is made possible by an **Analog-to-Digital Conversion** (**ADC**) process, which implies two key elements: ***sampling*** and ***quantization***.

- **Sampling** means taking a sample of a signal at a certain frequency/rate (***sample rate***).
  - **Audio CD**: 44.1 kHz
  - **Pro Audio**: 48/96 kHz
  - **MP3 (compressed)**: 320/256/192/128/96/64 kbps

Digital Signal

- To *digitize* a continuous signal implies ***discretizing it***. This is made possible by an **Analog-to-Digital Conversion** (**ADC**) process, which implies two key elements: ***sampling*** and ***quantization***.
- **Sampling** means taking a sample of a signal at a certain frequency/rate (***sample rate***).
  - **Audio CD**: 44.1 kHz
  - **Pro Audio**: 48/96 kHz
  - **MP3 (compressed)**: 320/256/192/128/96/64 kbps
- Because of ***aliasing***, the sampling rate must be ***at least two times superior to the highest frequency*** we want to represent (**Nyquist-Shannon**)

Digital Signal

- Once we take a sample of a signal at a given time, we need to determine the ***scale of its value***, this is called ***quantization***. Increasing the scale implies reducing the ***quantization noise*** (*quality vs. storage tradeoff*).

Digital Signal

- Once we take a sample of a signal at a given time, we need to determine the ***scale of its value***, this is called ***quantization***. Increasing the scale implies reducing the ***quantization noise*** (*quality vs. storage tradeoff*).

- **Audio CD**: 16-bits (65,536 values, 98 dB SNR)
- **Pro Audio**: 24-bits (~16,7 mil., 146 dB SNR)
- ***DSP***: 32/64-bits float (~4,3 bil., 194 dB SNR)

Digital Signal

- Once we take a sample of a signal at a given time, we need to determine the ***scale of its value***, this is called ***quantization***. Increasing the scale implies reducing the ***quantization noise*** (*quality vs. storage tradeoff*).

- **Audio CD**: 16-bits (65,536 values, 98 dB SNR)
- **Pro Audio**: 24-bits (~16,7 mil., 146 dB SNR)
- ***DSP***: 32/64-bits float (~4,3 bil., 194 dB SNR)

- For **DSP**, it is easier to make computations in ***floating-point*** (decimal), and *normalize* the signal between *-1.0* and *1.0*.

Digital Signal

- Once we take a sample of a signal at a given time, we need to determine the ***scale of its value***, this is called ***quantization***. Increasing the scale implies reducing the ***quantization noise*** (*quality vs. storage tradeoff*).

- **Audio CD**: 16-bits (65,536 values, 98 dB SNR)
- **Pro Audio**: 24-bits (~16,7 mil., 146 dB SNR)
- ***DSP***: 32/64-bits float (~4,3 bil., 194 dB SNR)

- For **DSP**, it is easier to make computations in ***floating-point*** (decimal), and *normalize* the signal between *-1.0* and *1.0*.

- Finally, sending a digital signal to ***audio speakers*** involves the inverse process of an **ADC**: *Digital-to-Analog Conversion* (**DAC**).

Faust (Functional Audio Stream)

Faust

- **FAUST** is a programming language specifically made for audio DSP and synthesis. It was created by **Yann Orlarey**, **Dominique Fober** & **Stéphane Letz** at **GRAME** in 2002.
  - **Functional** paradigm,
  - Declarative, ***math-like*** syntax
  - Produces **optimized code** for many architectures
  - Not recommended (yet) for multi-rate (**FFT**)
  - **Plugins**: VST, CLAP, AudioUnit
  - **Software**: **JUCE**, VCV Rack, Max, Pd, SuperCollider, Csound, …
  - **OS**: Linux, macOS, Windows, Android, iOS,
  - **Embedded**: Bela, Teensy, Daisy, ESP32, FPGA…
  - **Code**: **C/C++**, Rust, WASM, Java…

- **Tutorials** & **documentation** available at https://faust.grame.fr

- Dedicated online **IDE**: https:://faustide.grame.fr

Faust: Playground

https://faustplayground.grame.fr/

Faust: Basics

Faust: Basics

- Basic program: `import` and `process` statements.

Listing 1 `comments.dsp`

```
// Import statement:
// we import all of the faust libraries.
import("stdfaust.lib");

// 'process' statement:
// what your program will be processing/running:
process = 0;

// /!\ IMPORTANT:
// All the code expressions in Faust
// end with a semicolon ';'

```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.

Listing 2 `libraries.dsp`

```
// Import statement:
// we import all of the faust libraries.
import("stdfaust.lib");

// Here, we use the 'oscrc' function
// from the 'os' (oscillators) library:
process = os.oscrc(440) * 0.25;

```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.

- In **DSP**, we represent an *audio signal* with ***floating-point*** (decimal) numbers, and scale its values **between -1.0 and +1.0**.

Listing 3 `amplitude.dsp`

```
import("stdfaust.lib");

// We run the oscillators at 440Hz (A440),
// we multiply its amplitude by 0.25,
// which means we divide its volume by 4:
process = os.oscrc(440) * 0.25;

// Which would be the same as:
process = os.oscrc(440) / 4;

```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.
- In **DSP**, we represent an *audio signal* with ***floating-point*** (decimal) numbers, and scale its values **between -1.0 and +1.0**.

- **Mixing two signals** can be done using the **'+'** operator.

Listing 4 `mixing.dsp`

```
import("stdfaust.lib");

// Mixing two signals means adding them together,
// which can be done by using the '+' operator
process = (os.oscrc(440) + os.oscrc(880)) * 0.25;

// Here, we used parentheses because of
// operator precedence.
// Multiplications are always evaluated before
// additions or substractions.

```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.
- In **DSP**, we represent an *audio signal* with ***floating-point*** (decimal) numbers, and scale its values **between -1.0 and +1.0**.
- **Mixing two signals** can be done using the **'+'** operator.

- When **multiplying two signals** together, we can already make a really **simple DSP effect**: **Amplitude Modulation** (or **Ring Modulation**)

Listing 5 `ampmod.dsp`

```
import("stdfaust.lib");

// Basic amplitude modulation.
// no need for parentheses here, since we only
// use multiplications:
process = os.oscrc(440) * os.oscrc(10) * 0.25;
```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.

- In **DSP**, we represent an *audio signal* with ***floating-point*** (decimal) numbers, and scale its values **between -1.0 and +1.0**.
- **Mixing two signals** can be done using the **'+'** operator.
- When **multiplying two signals** together, we can already make a really **simple DSP effect**: **Amplitude Modulation** (or **Ring Modulation**)

- Faust can process **multiple channels**, by using the **','** symbol (**Parallel Operator**) to put signals **in parallel**.

Listing 6 `parallel.dsp`

```
import("stdfaust.lib");

// Here, we use the Parallel Operator ','
// to put two signals in parallel:
process = (os.oscrc(440) * 0.25), (os.oscrc(880) * 0.25);
```

Faust: Basics

- Basic program: `import` and `process` statements.
- Use a **Sinewave Oscillator** from the library.
- In **DSP**, we represent an *audio signal* with ***floating-point*** (decimal) numbers, and scale its values **between -1.0 and +1.0**.
- **Mixing two signals** can be done using the **'+'** operator.
- When **multiplying two signals** together, we can already make a really **simple DSP effect**: **Amplitude Modulation** (or **Ring Modulation**)
- Faust can process **multiple channels**, by using the **','** symbol (**Parallel Operator**) to put signals **in parallel**.

- We can declare **custom variables**, and **functions**.

Listing 7 `custom-functions.dsp`

```
import("stdfaust.lib");

// We can declare our custom functions:
osc440 = os.oscrc(440) * 0.25;
osc880 = os.oscrc(880) * 0.25;

// And call them from anywhere:
process = osc440, osc880;

```

Faust: Basics (2)

- In Faust, **connecting DSP functions** can be done using the ***sequential operator*** '**:**'.

Listing 8 `sequential.dsp`

```
import("stdfaust.lib");

// Connect A : B
// A must have the same number of outputs as
// the number of inputs of B.
process = os.sawtooth(440) : fi.lowpass(1, 440);
```

Faust: Basics (2)

- In Faust, **connecting DSP functions** can be done using the ***sequential operator*** '**:**'.
- The ***split operator*** '**<:**' and ***cable operator*** can also be used to connect a signal to **multiple targets**.

Listing 9 `split-cable.dsp`

```
import("stdfaust.lib");

// Connect A : B
// A must have the same number of outputs as
// the number of inputs of B
saw_filter = os.sawtooth(440) : fi.lowpass(1, 440);

// Connect A <: B
// The number of inputs of B must be a multiple
// of the number of outputs of A.
process =  saw_filter <: _,_;

```

Faust: Basics (2)

- In Faust, **connecting DSP functions** can be done using the ***sequential operator*** '**:**'.
- The ***split operator*** '**<:**' and ***cable operator*** can also be used to connect a signal to **multiple targets**.
- On the other hand, the ***merge operator*** '**:>**' can be used to **merge** (**mix**) **signals together**.

Listing 10 `examples/basics2/merge-cable.dsp`

```
import("stdfaust.lib");

// Equivalent to:
// osc = os.oscrc(440) + os.oscrc(880)
osc = (os.oscrc(440), os.oscrc(880)) :> _;

process =  osc * 0.25 <: _,_;

```

Faust: Basics (2)

- In Faust, **connecting DSP functions** can be done using the ***sequential operator*** '**:**'.
- The ***split operator*** '**<:**' and ***cable operator*** can also be used to connect a signal to **multiple targets**.

- On the other hand, the ***merge operator*** '**:>**' can be used to **merge** (**mix**) **signals together**.
- *Graphical User Interface* (**GUI**) elements can be added to ***control parameters***: *sliders*, *buttons*, *switches*, *vu-meters*, etc.

Listing 11 `examples/basics2/sliders.dsp`

```
import("stdfaust.lib");

// hslider("name", default, min, max, step)
saw_freq = hslider("saw_freq", 440, 20, 10000, 1);
cutoff = hslider("cutoff", 440, 20, 10000, 1);

// We can directly pass the slider variable names as parameters:
saw_filter = os.sawtooth(saw_freq) : fi.lowpass(1, cutoff);

process =  saw_filter <: _,_;

```

Faust: Basics (2)

- In Faust, **connecting DSP functions** can be done using the ***sequential operator*** '**:**'.
- The ***split operator*** '**<:**' and ***cable operator*** can also be used to connect a signal to **multiple targets**.
- On the other hand, the ***merge operator*** '**:>**' can be used to **merge** (**mix**) **signals together**.
- *Graphical User Interface* (**GUI**) elements can be added to ***control parameters***: *sliders*, *buttons*, *switches*, *vu-meters*, etc.
- They can be used to make a proper **gain control** in **dB** for example.

Listing 12 `examples/basics2/gain.dsp`

```
import("stdfaust.lib");

// We add the '[style:knob]' metadata,
// and we also specify the unit:
gain = hslider("gain[style:knob][unit:dB]", -6, -96, 6, 1) : ba.db2linear;

saw_freq = hslider("saw_freq[unit:Hz]", 440, 20, 10000, 1);
cutoff = hslider("cutoff[unit:Hz]", 440, 20, 10000, 1);

saw_filter = os.sawtooth(saw_freq) : fi.lowpass(1, cutoff);

process =  saw_filter * gain <: _,_;

```

Faust: Functions

Faust: Functions

- ***Functions*** in programming languages usually consist in pieces of code containing a ***series of instructions to execute***. They can be called *once or multiple times*, *with or without variable parameters*, and can *return values*.

Faust: Functions

- ***Functions*** in programming languages usually consist in pieces of code containing a ***series of instructions to execute***. They can be called *once or multiple times*, *with or without variable parameters*, and can *return values*.
- ***Function definitions*** in Faust have the syntax *function(parameter1, parameter2, ...) = expression;*".

Listing 13 `faust/examples/functions/f1.dsp`

```
// Here, we define the function 'add',
// which takes 2 arguments (or parameters)
// that we call 'a' and 'b'.
// Note: 'a' and 'b' are arbitrary names,
// we could have used any name we wanted.
add(a, b) = a + b;

```

Faust: Functions

- ***Functions*** in programming languages usually consist in pieces of code containing a ***series of instructions to execute***. They can be called *once or multiple times*, *with or without variable parameters*, and can *return values*.
- ***Function definitions*** in Faust have the syntax *function(parameter1, parameter2, ...) = expression;*".

- In order to ***call*** (**execute**) that function, we need to ***replace its parameters (arguments)*** with the values we want to pass as parameters.

Listing 14 `faust/examples/functions/f2.dsp`

```
add(a, b) = a + b;

// If we want to call (execute) that function,
// we need to replace 'a' & 'b' by the parameters
// we want to pass to the function.
// Here, we also store the result into a 'variable' called "result".
result = add(1, 2);

// Under the hood, 'a' and 'b' are replaced by '1' and '2':
add(1, 2) = 1 + 2;

```

Faust: Functions

- ***Functions*** in programming languages usually consist in pieces of code containing a ***series of instructions to execute***. They can be called *once or multiple times*, *with or without variable parameters*, and can *return values*.

- ***Function definitions*** in Faust have the syntax *function(parameter1, parameter2, …) = expression;*".
- In order to ***call*** (**execute**) that function, we need to ***replace its parameters (arguments)*** with the values we want to pass as parameters.

- Arguments are only valid inside of the function definition, we say that they are *local to the **scope** of the function*. This also means that they ***take precedence*** over any other variable or expression that have the same name in the code.

Listing 15 `faust/examples/functions/f3.dsp`

```
a = 31;
b = 47;

add(a, b) = a + b;
result = add(1, 2); // result will be 3, not 78!

// On the other hand, if we were to change
// the name of the arguments of the 'add' function, like so:
add(c, d) = a + b;
// The result would be 78 this time,
// because 'a' and 'b' now refer to
// the 'a = 31' and 'b = 47' expressions.

```

Faust: Functions (2)

- In Faust, everything is a ***function of time***. When a function is called in a Faust program, it will be called ***for each computation of a sample***, *e.g. 44100 times per second.*

Listing 16 `faust/examples/functions-signals/f0.dsp`

```
// Example of a constant function,
// it will produce the same output no matter
// how many times it is called:
my_constant_function = 440;

// We define a function 'oscillator'
// which itself calls the 'os.osci' function.
// os.osci computes for each new sample in time
// a new sinewave value.
oscillator = os.osci(440);

// Here, we sum the values of the two 'os.osci'
// functions, sample-by-sample:
mix = os.osci(440) + os.osci(880);

```

Faust: Functions (2)

- In Faust, everything is a ***function of time***. When a function is called in a Faust program, it will be called ***for each computation of a sample***, *e.g. 44100 times per second.*
- In the end, all functions produces a numerical value (integer or floating point). Therefore, anything can be passed as a ***function argument***...

Listing 17 `faust/examples/functions/f4.dsp`

```
// We can pass anything as a function argument in Faust:
add(a, b) = a + b;

// We call 'add' on two sinewave oscillators
// with different frequencies.
// This will sum the two signals together:
process = add(os.osci(440), os.osci(880));

// It also works for GUI elements, like sliders:
slider_a = hslider("slider_a", 5, 0, 10, 1);
slider_b = hslider("slider_b", 5, 0, 5, 1);

process = add(slider_a, slider_b);

```

Faust: Functions (2)

- In Faust, everything is a ***function of time***. When a function is called in a Faust program, it will be called ***for each computation of a sample***, *e.g. 44100 times per second.*
- In the end, all functions produces a numerical value (integer or floating point). Therefore, anything can be passed as a ***function argument***…
- …***including functions***!

Listing 18 `faust/examples/functions/f5.dsp`

```
// And... other functions!
// Here we declare the function 'osc_a',
// that takes on argument called 'freq',
// which we are going to pass through to the os.osci function.
// And yes 'os.osci' is also a function, which takes one argument (frequency)
// and produces a sinewave at that frequency.
osc_a(freq) = os.osci(freq);

// Same here with a sawtooth, the name 'freq' doesn't conflict with the one
// from osc_a because they're only valid within their own scope:
osc_b(freq) = os.sawtooth(freq);

// We can call them like this:
process = osc_a(440) + osc_b(880);

// And what happens under the hood, if we unroll the function calls, will be this:
osc_a(440) = os.osci(440);
osc_b(880) = os.sawtooth(880);

// We can also do the same thing in function calls:
add(a, b) = a + b;
osc_a(freq) = os.osci(freq);
osc_b(freq) = os.sawtooth(freq);

process = add(osc_a(440), osc_b(880));

// If we unroll the calls, we get this:
osc_a(440) = os.osci(440);
osc_b(880) = os.sawtooth(880);
add(osc_a(440), osc_b(880)) = osc_a(440) + osc_b(880);
```

Faust: Functions (2)

Listing 19 `faust/examples/functions/f5.dsp`

```
// And... other functions!
// Here we declare the function 'osc_a',
// that takes on argument called 'freq',
// which we are going to pass through to the os.osci function.
// And yes 'os.osci' is also a function, which takes one argument (frequency)
// and produces a sinewave at that frequency.
osc_a(freq) = os.osci(freq);

// Same here with a sawtooth, the name 'freq' doesn't conflict with the one
// from osc_a because they're only valid within their own scope:
osc_b(freq) = os.sawtooth(freq);

// We can call them like this:
process = osc_a(440) + osc_b(880);

// And what happens under the hood, if we unroll the function calls, will be this:
osc_a(440) = os.osci(440);
osc_b(880) = os.sawtooth(880);

// We can also do the same thing in function calls:
add(a, b) = a + b;
osc_a(freq) = os.osci(freq);
osc_b(freq) = os.sawtooth(freq);

process = add(osc_a(440), osc_b(880));

// If we unroll the calls, we get this:
osc_a(440) = os.osci(440);
osc_b(880) = os.sawtooth(880);
add(osc_a(440), osc_b(880)) = osc_a(440) + osc_b(880);

```

Faust: Synthesis

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*

Listing 20 `faust/examples/synthesis/oscillators.dsp`

```
import("stdfaust.lib");

sine = os.osci(440) * 0.25;
triangle = os.triangle(440) * 0.25;
sawtooth = os.sawtooth(440) * 0.25;
square = os.square(440) * 0.25;

process =  sine;
// process = triangle;
// process = sawtooth;
// process = square;

```

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*

- When a Faust program starts to be a little more complex, its always good practice to ***refactor code*** by using ***custom functions*** with variable **parameters**.

Listing 21 `faust/examples/synthesis/functions.dsp`

```
import("stdfaust.lib");

// Declaring functions, with variable parameters:
sine(freq) = os.osci(freq);
triangle(freq) = os.triangle(freq);
sawtooth(freq) = os.sawtooth(freq);
square(freq) = os.square(freq);

// Passing arguments from function to function:
synth(osc, freq, gain) = osc(freq) * gain;

// This way, we can modify everything here:
process =  synth(sine, 440, ba.db2linear(-16));

```

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*
- When a Faust program starts to be a little more complex, its always good practice to ***refactor code*** by using ***custom functions*** with variable **parameters**.

- In Faust, **functions** can take ***any element of the language as parameters***, including **GUI elements**.

Listing 22 `faust/examples/synthesis/functions-gui.dsp`

```
import("stdfaust.lib");

sine(freq) = os.osci(freq);
triangle(freq) = os.triangle(freq);
sawtooth(freq) = os.sawtooth(freq);
square(freq) = os.square(freq);

synth(osc, freq, gain) = osc(freq) * gain : fi.lowpass(1, freq);

// Declare some controls:
freq_gui = nentry("frequency[unit:Hz]", 440, 20, 1000, 1);
gain_gui = hslider("gain[style:knob][unit:dB]", -16, -32, 0, 1) : ba.db2linear;

// We can pass GUI elements to a function:
process =  synth(square, freq_gui, gain_gui);

```

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*
- When a Faust program starts to be a little more complex, its always good practice to ***refactor code*** by using ***custom functions*** with variable **parameters**.
- In Faust, **functions** can take ***any element of the language as parameters***, including **GUI elements**.

- Finally, the ***select*** primitive (an equivalent to ***switch*** in Max), allow to select an input from a list. It can be used in this case to *switch between* ***waveforms***

Listing 23 `faust/examples/synthesis/select.dsp`

```
import("stdfaust.lib");

wave1 = os.triangle(440);
wave2 = os.square(440);

switch_gui = nentry("waveform", 0, 0, 1, 1);

process = wave1, wave2 : select2(switch_gui) * 0.25;

```

Faust: Synthesis

- Faust has a in its libraries a good collection of '*basic*' **oscillators**, with different ***waveforms***: *sine*, *triangle*, *sawtooth*, *square*, *etc.*
- When a Faust program starts to be a little more complex, its always good practice to ***refactor code*** by using ***custom functions*** with variable **parameters**.
- In Faust, **functions** can take ***any element of the language as parameters***, including **GUI elements**.
- Finally, the ***select*** primitive (an equivalent to ***switch*** in Max), allow to select an input from a list. It can be used in this case to *switch between* ***waveforms***

- Our goal now will be to ***apply this to our previous synthesizer.***

Listing 24 `faust/examples/synthesis/switch-waveform.dsp`

```
import("stdfaust.lib");

sine(freq) = os.osci(freq);
triangle(freq) = os.triangle(freq);
sawtooth(freq) = os.sawtooth(freq);
square(freq) = os.square(freq);

oscillator(freq) = sine(freq), triangle(freq), sawtooth(freq), square(freq);

switch_gui  = nentry(
    "waveform[style:menu{'Sine':0; 'Triangle':1; 'Sawtooth':2; 'Square':3}]",
    0, 0, 3, 1
);

switch(freq) = oscillator(freq) : ba.selectn(4, switch_gui);

freq_gui = nentry("frequency[unit:Hz]", 440, 20, 1000, 1);
gain_gui = hslider("gain[style:knob][unit:dB]", -16, -32, 0, 1) : ba.db2linear;

synth(freq, gain) = switch(freq) * gain : fi.lowpass(1, freq);

process =  synth(freq_gui, gain_gui);

```

Faust: Synthesis

Listing 25 `faust/examples/synthesis/switch-waveform.dsp`

```
import("stdfaust.lib");

sine(freq) = os.osci(freq);
triangle(freq) = os.triangle(freq);
sawtooth(freq) = os.sawtooth(freq);
square(freq) = os.square(freq);

oscillator(freq) = sine(freq), triangle(freq), sawtooth(freq), square(freq);

switch_gui  = nentry(
    "waveform[style:menu{'Sine':0; 'Triangle':1; 'Sawtooth':2; 'Square':3}]",
    0, 0, 3, 1
);

switch(freq) = oscillator(freq) : ba.selectn(4, switch_gui);

freq_gui = nentry("frequency[unit:Hz]", 440, 20, 1000, 1);
gain_gui = hslider("gain[style:knob][unit:dB]", -16, -32, 0, 1) : ba.db2linear;

synth(freq, gain) = switch(freq) * gain : fi.lowpass(1, freq);

process =  synth(freq_gui, gain_gui);

```

Faust: Filtering

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.

- In this first example, we use a **lowpass filter** to *cut-off* all frequencies **above** the one specified with the slider.

Listing 26 `faust/examples/filtering/sawtooth-lowpass.dsp`

```
import("stdfaust.lib");

cutoff = hslider("cutoff[unit:Hz]", 200, 20, 5000, 1);

process = os.sawtooth(220) : fi.lowpass(2, cutoff);

```

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.
- In this first example, we use a **lowpass filter** to *cut-off* all frequencies **above** the one specified with the slider.
- Inversely, we can use a **highpass filter** to filter out frequencies **below** a certain threshold.

Listing 27 `faust/examples/filtering/sawtooth-highpass.dsp`

```
import("stdfaust.lib");

cutoff = hslider("cutoff[unit:Hz]", 1000, 20, 5000, 1);

process = os.sawtooth(220) : fi.highpass(2, cutoff);

```

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.
- In this first example, we use a **lowpass filter** to *cut-off* all frequencies **above** the one specified with the slider.
- Inversely, we can use a **highpass filter** to filter out frequencies **below** a certain threshold.
- … or specify a *restrictive frequency band*, using a **bandpass filter**.

Listing 28 `faust/examples/filtering/sawtooth-bandpass.dsp`

```
import("stdfaust.lib");

cutoff = hslider("cutoff[unit:Hz]", 1000, 20, 5000, 1);

process = os.sawtooth(220) : fi.bandpass(1, cutoff-100, cutoff+100);

```

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.
- In this first example, we use a **lowpass filter** to *cut-off* all frequencies **above** the one specified with the slider.
- Inversely, we can use a **highpass filter** to filter out frequencies **below** a certain threshold.
- … or specify a *restrictive frequency band*, using a **bandpass filter**.

- In general, filters are applied **progressively**, the frequency *decay* (**roll-off**) depends on the **filter order**.

Listing 29 `faust/examples/filtering/sawtooth-lowpass-order.dsp`

```
import("stdfaust.lib");

cutoff = hslider("cutoff[unit:Hz]", 200, 20, 5000, 1);

// Compile-time constant,
// Cannot be used with a slider:
ORDER = 1;

process = os.sawtooth(220) : fi.lowpass(ORDER, cutoff);

```

Faust: Filtering

- **Filtering** is the process of **altering** the *spectrum* of a signal using a **transfer function**. A filter can *remove* or *attenuate* specific frequencies, and/or *boost* others.
- In this first example, we use a **lowpass filter** to *cut-off* all frequencies **above** the one specified with the slider.

- Inversely, we can use a **highpass filter** to filter out frequencies **below** a certain threshold.
- … or specify a *restrictive frequency band*, using a **bandpass filter**.

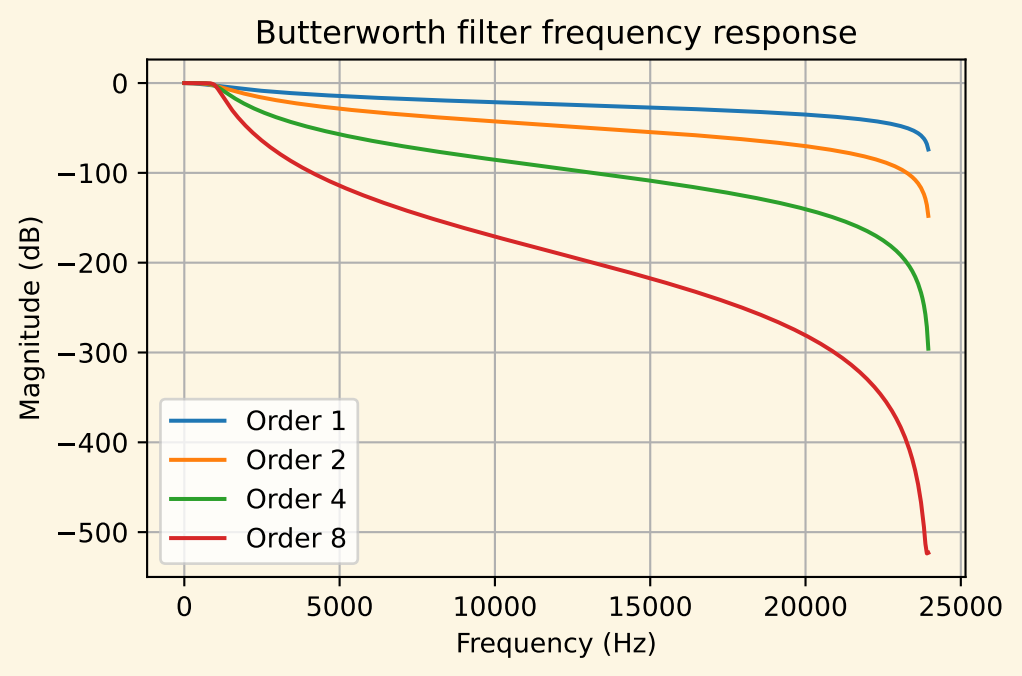
- In general, filters are applied **progressively**, the frequency *decay* (**roll-off**) depends on the **filter order**.

```
import numpy as np;
import matplotlib.pyplot as plt;
from scipy import signal;

fs = 48000;

b1, a1 = signal.butter(1, 1000, btype='low', fs=fs);
b2, a2 = signal.butter(2, 1000, btype='low', fs=fs);
b4, a4 = signal.butter(4, 1000, btype='low', fs=fs);
b8, a8 = signal.butter(8, 1000, btype='low', fs=fs);
w1, h1 = signal.freqz(b1, a1, fs=fs);
w2, h2 = signal.freqz(b2, a2, fs=fs);
w4, h4 = signal.freqz(b4, a4, fs=fs);
w8, h8 = signal.freqz(b8, a8, fs=fs);

# Filter frequency response:
plt.figure(facecolor="#fdf6e3")
ax = plt.gca()
ax.set_facecolor("#fdf6e3")
plt.grid();
plt.plot(w1, 20 * np.log10(np.abs(h1)), label="Order 1");
plt.plot(w2, 20 * np.log10(np.abs(h2)), label="Order 2");
plt.plot(w4, 20 * np.log10(np.abs(h4)), label="Order 4");
plt.plot(w8, 20 * np.log10(np.abs(h8)), label="Order 8");
plt.xlabel("Frequency (Hz)");
plt.ylabel("Magnitude (dB)");
plt.title("Butterworth filter frequency response");
plt.legend();
plt.show();
```



Faust: Filtering

- **Faust** has quite a large range of filters (`fi` library prefix) with different behaviors and responses, from basic *one-pole/one-zero* **building blocks** to full parametric **equalizers** or octave **filter-banks**.

Faust: Compression

Faust: Compression

- **Compression** is the process of **reducing the dynamic range** of a signal using a **transfer function**. Compressors are usually used to **dynamically manage the volume** of a specific sound source, in order to **improve consistency during mixing and mastering**.

Faust: Compression

- **Compression** is the process of **reducing the dynamic range** of a signal using a **transfer function**. Compressors are usually used to **dynamically manage the**

volume of a specific sound source, in order to **improve consistency during mixing and mastering**.

- **Control parameters** are usually the following:
  - **Threshold** (dB): the level above which the amplitude of a signal is going to be reduced.
  - **Ratio**: the reduction factor when amplitude is above the threshold.
  - **Attack** (ms): the speed at which compression is applied.
  - **Release** (ms): the speed at which compression is released.
  - **Makeup gain** (dB): final gain applied to the compressed signal.

Faust: Time

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).

Listing 30 `faust/examples/time/recursive.dsp`

```
// We connect the signal '1' and an empty signal
// to the operator '+'.
// This expression has exactly one input
// and one output, same for an empty signal.
// Therefore, the two expressions can be
// mutually recursive:
process = (_ + 1) ~ _;
//        (  A  ) ~ B

```

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).

Interactive version (values on each wire, sample by sample): Faust Stepper

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).
- The recursive operator is typically used to implement **counters**, and/or ***to represent time***. With it, we can count for instance *up to 44100 or 48000 samples* to ***represent one second of audio time***.

Listing 31 `faust/examples/time/counter_samples.dsp`

```
import("stdfaust.lib");

// We could use our previous definition
// of the counter, and wrap it around the
// value of the sample rate (using modulo)
// But there's going to be something wrong
// with it...
increment = (_ + 1) ~ _;
bad_counter = increment % 48000;

// This solution works:
counter = ((_ + 1) % 48000) ~ _;
//        (     A         ) ~ B

// We can put all of this in a function
// and make the limit of the counter variable:
counter(n) = ((_ + 1) % n) ~ _;

process = counter(ma.SR);

```

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).
- The recursive operator is typically used to implement **counters**, and/or ***to represent time***. With it, we can count for instance *up to 44100 or 48000 samples* to ***represent one second of audio time***.

Interactive version (values on each wire, sample by sample): Faust Stepper

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).
- The recursive operator is typically used to implement **counters**, and/or ***to represent time***. With it, we can count for instance *up to 44100 or 48000 samples* to ***represent one second of audio time***.

- When implementing ***time-related functions***, it can prove very useful to ***monitor values from the GUI***.

Listing 32 `faust/examples/time/monitoring_values.dsp`

```
import("stdfaust.lib");

SR = 48000;
gui = vbargraph("value[style:numerical]", 0, SR);
counter(n) = ((_ + 1) % n) ~ _;

process = counter(SR): gui;

```

Faust: Time (1/2)

- Faust has one final composition operator: the ***recursive operator*** (‘~’) which allows to ***connect two signals recursively*** (with ***a delay of one sample***).
- The recursive operator is typically used to implement **counters**, and/or ***to represent time***. With it, we can count for instance *up to 44100 or 48000 samples* to ***represent one second of audio time***.
- When implementing ***time-related functions***, it can prove very useful to ***monitor values from the GUI***.

- We can for example use a counter to ***switch our synth’s waveform automatically every second.***

Listing 33 `faust/examples/time/waveform_switch_counter.dsp`

```
import("stdfaust.lib");

SR = 48000;
gui = vbargraph("value[style:numerical]", 0, SR);

// We make the '1' (increment) variable:
counter(i, n) = ((_ + i) % n) ~ _;

// Outputs the value '1'
// whenever the counter reaches 48000
// (every second):
count1s = counter(1, SR) == 0;

process = counter(count1s, 4) : gui;

```

Faust: Time (2/2)

- A counter can be used as a ***base for many many things***. With it, we can for instance build a ***ramp signal*** that *goes from 0 to 1 repeatedly at a certain rate*.

Listing 34 `faust/examples/time/phasor.dsp`

```
import("stdfaust.lib");

counter(i, n) = ((_ + i) % n) ~ _;

// We can put a decimal as increment value,
// and wrap it around 0.0 and 1.0:
process = counter(0.00195, 1.0);

// But it would be much better to
// use the frequency unit in Hertz,
// We know one second of time is the sample rate,
// so to get the increment value, we can use:
frequency = 440/ma.SR;

process = counter(frequency, 1.0);

```

Faust: Time (2/2)

- A counter can be used as a base for many many things. With it, we can for instance build a ***ramp signal*** that *goes from 0 to 1 repeatedly at a certain rate.*
- Notice anything? A *ramp* is very much like a ***sawtooth oscillator***, only with a different range (0 to 1 instead of -1 to 1).

Listing 35 `faust/examples/time/phasor.dsp`

```
import("stdfaust.lib");

counter(i, n) = ((_ + i) % n) ~ _;

// We can put a decimal as increment value,
// and wrap it around 0.0 and 1.0:
process = counter(0.00195, 1.0);

// But it would be much better to
// use the frequency unit in Hertz,
// We know one second of time is the sample rate,
// so to get the increment value, we can use:
frequency = 440/ma.SR;

process = counter(frequency, 1.0);

```

Faust: Time (2/2)

- A counter can be used as a base for many many things. With it, we can for instance build a ***ramp signal*** that *goes from 0 to 1 repeatedly at a certain rate.*
- Notice anything? A *ramp* is very much like a ***sawtooth oscillator***, only with a different range (0 to 1 instead of -1 to 1).

- With a *ramp* (or *phasor*), we can pretty much already build ***all of the simple waveform oscillators***, with a few operations…

Listing 36 `faust/examples/time/waveforms.dsp`

```
import("stdfaust.lib");

counter(i, n) = ((_ + i) % n) ~ _;
phasor(f) = counter(f/ma.SR, 1.0);

// For sawtooth, we re-scale the signal to [-1, 1]
sawtooth(f) = phasor(f) * 2.0 - 1.0;

// Sine function has an input range
// of [0, 2*PI]:
sine(f) = sin(phasor(f) * 2.0 * ma.PI);

// For square, we can use a simple 'if' expression
// output -1.0 when phase is below 0.5
// and 1.0 when it's above:
square(f) = ba.if(
    phasor(f) > 0.5, // condition
    1.0, // then
   -1.0  // else
);

// Triangle is a little more complicated,
// we need to invert the signal when it
// reaches half of its max amplitude,
// and then rescale it to [-1, 1]:
triangle(f) =
    (
        ba.if(
            phasor(f) < 0.5,
            phasor(f),
            1.0-phasor(f)
        )
        - 0.25
    )
    * 4.0;

process = phasor(440) * 0.25;

```

Faust: Buffers & Delays

Faust: Buffers & Delays

- *Ramps* are often used as ***cursors*** to *read* or *write samples* in a ***buffer***. In Faust, *buffers* are implemented with the `rdtable` & `rwtable` primitives.

Listing 37 `faust/examples/time/tables.dsp`

```
import("stdfaust.lib");

counter(i, n) = ((_ + i) % n) ~ _;

// We define a maximum length for our buffer:
BUFFER_MAX = 48000 * 5;

// r: read position in samples.
// w: write position in samples.
// x: input signal to write.
buffer(r, w, x) = rwtable(BUFFER_MAX, 0.0, w, x, r);

// Define our read/write cursors:
record = counter(button("record"), BUFFER_MAX)
       : vbargraph("vrec[style:numerical]", 0, BUFFER_MAX);

play = counter(button("play"), BUFFER_MAX)
     : vbargraph("vplay[style:numerical]", 0, BUFFER_MAX);

// And we have now a (very) simple looper:
process = buffer(play, record, _);


```

Faust: Buffers & Delays

- *Ramps* are often used as ***cursors*** to *read* or *write samples* in a ***buffer***. In Faust, *buffers* are implemented with the `rdtable` & `rwtable` primitives.
- To playback a ***sound file***, we also typically use a phasor as a *read cursor*.

Listing 38 `faust/examples/time/soundfile.dsp`

```
import("stdfaust.lib");

// We declare a root directory,
// from which we can retrieve a
// collection of sound files:
declare soundfiles "https://raw.githubusercontent.com/sletz/faust-sampler/main";

// We declare a soundfile with one output (mono);
violin = soundfile("sound[url:{'violon.wav'}]", 1) : (!,!,_);

// Play for 5 seconds:
length = ma.SR * 5;

counter(i, n) = ((_ + i) % n) ~ _;
sampler(s) = 0, counter(s, length) : violin;

process = sampler(1);

```

Faust: Buffers & Delays

- *Ramps* are often used as ***cursors*** to *read* or *write samples* in a ***buffer***. In Faust, *buffers* are implemented with the `rdtable` & `rwtable` primitives.
- To playback a ***sound file***, we also typically use a phasor as a *read cursor*.

- **Delaying** a signal in Faust can be done using the operator '`@`'. Thus, the expression ***x*** `@` ***n*** delays a signal '`x`' by '`n`' **samples**.

Listing 39 `faust/examples/time/delays.dsp`

```
import("stdfaust.lib");

// Simple sine oscillator at low frequency:
osc = os.osci(110);

// We use an ADSR envelope, to make it percussive:
env = button("play") : en.adsr(
    0.0125, // attack time (s)
    0.25, // decay time (s)
    0.0, // sustain level
    0.0 // release time
);

// Define our stereo delay,
// with constant delay times:
del(x) =
    x + x @ ba.sec2samp(0.5),
    x + x @ ba.sec2samp(0.25);

process = osc * env : del;

```

Faust: Buffers & Delays

- *Ramps* are often used as ***cursors*** to *read* or *write samples* in a ***buffer***. In Faust, *buffers* are implemented with the `rdtable` & `rwtable` primitives.
- To playback a ***sound file***, we also typically use a phasor as a *read cursor*.
- **Delaying** a signal in Faust can be done using the operator '`@`'. Thus, the expression ***x @ n*** delays a signal 'x' by 'n' **samples**.

- ***Delay times*** can usually be changed in real time from **GUI**, but it will in this case introduce unwanted '*clicks*'. We also here define a '***Dry/Wet***' control.

Listing 40 `faust/examples/time/delays-sliders.dsp`

```
import("stdfaust.lib");

// Simple sine oscillator at low frequency:
osc = os.osci(110);

// We use an ADSR envelope, to make it percussive:
env = button("play") : en.adsr(
    0.0125, // attack time (s)
    0.25, // decay time (s)
    0.0, // sustain level
    0.0 // release time
);

del_l = hslider("Delay (left)", 0.25, 0, 5, 0.1) : ba.sec2samp;
del_r = hslider("Delay (right)", 0.5, 0, 5, 0.1) : ba.sec2samp;
drywet = hslider("Dry/Wet [style:knob]", 0.5, 0, 1, 0.01);

// Define our stereo delay:
del(x) =
    x * (1-drywet) + (x @ del_l) * drywet,
    x * (1-drywet) + (x @ del_r) * drywet;

process = osc * env : del;

```

Faust: Buffers & Delays

- *Ramps* are often used as ***cursors*** to *read* or *write samples* in a ***buffer***. In Faust, *buffers* are implemented with the `rdtable` & `rwtable` primitives.
- To playback a ***sound file***, we also typically use a phasor as a *read cursor*.
- **Delaying** a signal in Faust can be done using the operator '`@`'. Thus, the expression ***`x @ n`*** delays a signal 'x' by 'n' **samples**.
- ***Delay times*** can usually be changed in real time from **GUI**, but it will in this case introduce unwanted '*clicks*'. We also here define a '***Dry/Wet***' control.

- Another typical control parameter for delays is the '***feedback coefficient***', which allows the delay to *repeat itself a certain number of times* before fading away. This will require to ***use the recursive operator***.

Listing 41 `faust/examples/time/delays-feedback.dsp`

```
import("stdfaust.lib");

osc = os.osci(440);

// n: delay length (in samples)
// f: feedback coeff. [0, 1]
// x: input signal to be delayed
dfb(n, f, x) = (x + (_ * f)) @ n ~ _;

// We use an ADSR envelope, to make it percussive:
env = button("play") : en.adsr(
    0.0125, // attack time (s)
    0.25, // decay time (s)
    0.0, // sustain level
    0.0 // release time
);

del_l = hslider("Delay (left)", 0.25, 0, 5, 0.1) : ba.sec2samp;
del_r = hslider("Delay (right)", 0.5, 0, 5, 0.1) : ba.sec2samp;
fbk_l = hslider("Feedback (left)", 0.5, 0, 0.9, 0.01);
fbk_r = hslider("Feedback (right)", 0.5, 0, 0.9, 0.01);
drywet = hslider("Dry/Wet [style:knob]", 0.5, 0, 1, 0.01);

// Define our stereo delay:
del(x) = x * (1-drywet) + dfb(del_l, fbk_l, x) * drywet,
         x * (1-drywet) + dfb(del_r, fbk_r, x) * drywet;

process = osc * env : del;

```

Faust: Advanced Synthesis

Faust: Advanced Synthesis (1/2)

- *Waveform or Wavetable synthesis* is not the only method that we can use in order to create sound. We can also use ***additive synthesis***, which is a superposition of oscillators running at different frequencies. One really fast way to do this in Faust is to use the ***par*** operator, which puts signal expressions ***in parallel*** dynamically.

Listing 42 `faust/examples/synthesis/additive.dsp`

```
import("stdfaust.lib");

// Use the 'par' primitive, which allows to
// programatically put signal expressions
// in parallel.
sine8 = par(n, 8, os.osci(220*(n+1)));
process = sine8 :> _ * 1/16 <: _,_;

// OR we could list the frequencies that we want manually:
frequencies = (220, 440, 550, 551, 880, 1200);

// And connect to a parallel group of sine oscillators:
process = frequencies
        : par(i, 6, os.osci)
        :> _
        : /(12)
        <: _,_;

```

Faust: Advanced Synthesis (1/2)

- *Waveform or Wavetable synthesis* is not the only method that we can use in order to create sound. We can also use ***additive synthesis***, which is a superposition of oscillators running at different frequencies. One really fast way to do this in Faust is to use the ***par*** operator, which puts signal expressions ***in parallel*** dynamically.
- In most synthesizers, additive synthesis is done by ***mixing a few oscillators together***, with *selectable waveforms*.

Listing 43 `faust/examples/synthesis/additive-classic.dsp`

```
import("stdfaust.lib");

declare author "Pierre Cochard";

sine(f) = os.osci(f);
triangle(f) = os.triangle(f);
sawtooth(f) = os.sawtooth(f);
square(f) = os.square(f);

onoff = checkbox("on/off");

switch_gui(n) = nentry(
    "waveform %n[style:menu{
        'Sine':0; 'Triangle':1;
        'Sawtooth':2; 'Square':3
    }]",
    0, 0, 3, 1
);

freq(n) = hslider("Osc. %n frequency", 60, 0, 127, 1)
        : ba.midikey2hz;

gain(n) = hslider("Osc. %n amplitude", 0, 0, 1, 0.01);

voice(n, f) = sine(f), triangle(f), sawtooth(f), square(f)
            : ba.selectn(4, switch_gui(n));

process = par(n, 3, voice(n, freq(n)) * gain(n))
        :> _        // merge
        : *(1/3)    // div. volume
        <: _,_      // expand to stereo
;

```

Faust: Advanced Synthesis (1/2)

- *Waveform or Wavetable synthesis* is not the only method that we can use in order to create sound. We can also use ***additive synthesis***, which is a superposition of oscillators

running at different frequencies. One really fast way to do this in Faust is to use the ***par*** operator, which puts signal expressions ***in parallel*** dynamically.

- In most synthesizers, additive synthesis is done by ***mixing a few oscillators together***, with *selectable waveforms*.

- Usually, ***filtering*** is applied afterwards, to remove unwanted frequencies (***subtractive synthesis***).

Listing 44 `faust/examples/synthesis/subtractive.dsp`

```
import("stdfaust.lib");

sine(f) = os.osci(f);
triangle(f) = os.triangle(f);
sawtooth(f) = os.sawtooth(f);
square(f) = os.square(f);

switch_gui(n) = nentry(
    "waveform %n[style:menu{
        'Sine':0; 'Triangle':1;
        'Sawtooth':2; 'Square':3
        }]",
    0, 0, 3, 1
);

freq(n) = hslider("Osc. %n frequency", 60, 0, 127, 1)
        : ba.midikey2hz;

gain(n) = hslider("Osc. %n amplitude", 0, 0, 1, 0.01);

voice(n, f) = sine(f), triangle(f), sawtooth(f), square(f)
        : ba.selectn(4, switch_gui(n));

moog = ve.moog_vcf(
    hslider("moog_vcf:res", 0.5, 0, 1, 0.01),
    hslider("moog_vcf:freq", 1000, 50, 4000, 1)
);

process = par(n, 3, voice(n, freq(n)) * gain(n))
        :> _        // merge
        : *(1/3)    // div. volume
        : moog
        <: _,_      // expand to stereo
;

```

Faust: Advanced Synthesis (1/2)

- *Waveform or Wavetable synthesis* is not the only method that we can use in order to create sound. We can also use ***additive synthesis***, which is a superposition of oscillators running at different frequencies. One really fast way to do this in Faust is to use the ***par*** operator, which puts signal expressions ***in parallel*** dynamically.
- In most synthesizers, additive synthesis is done by ***mixing a few oscillators together***, with *selectable waveforms*.
- Usually, ***filtering*** is applied afterwards, to remove unwanted frequencies (***subtractive synthesis***).

- **Frequency-modulation** (***FM***) **synthesis** is another well-known synthesis method, which implies *inter-modulated oscillator graphs*.

Listing 45 `faust/examples/synthesis/fm.dsp`

```
import("stdfaust.lib");

// We modulate the frequency of an oscillator by another:
minimal_fm(f,m) = os.osci(
    f + f * os.osci(m)
);

process = minimal_fm(440, 20);

// But it's better to use the one in the library:
process = sy.fm((440, 500, 800), (500, 400)) * 0.25 <: _,_;

```

Faust: Advanced Synthesis (2/2)

- ***Physical model synthesis*** consists in ***emulating the physical properties of an instrument*** in order to reproduce its sound. Different methods exist in order to achieve this: **modal synthesis**, **FDTD**, **digital waveguides**, *etc.*

Listing 46 `faust/examples/synthesis/pm.dsp`

```
import("stdfaust.lib");

// Simple Karplus-Strong:
process = ba.pulse(10000) :
        + ~ transformation;

transformation =
    @(hslider("delay", 128, 0, 200, 1))
    : moyenne
    : *(hslider("gain", 0.98, -0.98, 0.98, 0.01));

moyenne(x) = (x+x')/2;

```

Faust: Advanced Synthesis (2/2)

- ***Physical model synthesis*** consists in ***emulating the physical properties of an instrument*** in order to reproduce its sound. Different methods exist in order to achieve this: **modal synthesis**, **FDTD**, **digital waveguides**, *etc.*
- Faust has in its ***libraries*** ('`pm`' prefix) a good collection of ***physically-modeled instruments*** (*flutes*, *bells*, *guitars*, *voice*, *etc.*).

Listing 47 `faust/examples/synthesis/pm2.dsp`

```
import("stdfaust.lib");
// https://faustlibraries.grame.fr/libs/physmodels

// Guitars:
guitar1 = pm.elecGuitar_ui_MIDI;
guitar2 = pm.nylonGuitar_ui_MIDI;
guitar3 = pm.guitar_ui_MIDI;

// Orchestral:
violin = pm.violin_ui_MIDI;
clarinet = pm.clarinet_ui_MIDI;
brass = pm.brass_ui_MIDI;
flute = pm.flute_ui_MIDI

// Percussions:
djembe = pm.djembe_ui_MIDI;
marimba = pm.marimba_ui_MIDI;

// Bells:
bell1 = pm.churchBell_ui;
bell2 = pm.englishBell_ui;
bell3 = pm.frenchBell_ui;
bell4 = pm.germanBell_ui;
bell5 = pm.russianBell_ui;
bell6 = pm.standardBell_ui;

// Voice:
vocal = pm.SFFormantModelBP_ui;

```

Faust: Advanced Synthesis (2/2)

- ***Physical model synthesis*** consists in ***emulating the physical properties of an instrument*** in order to reproduce its sound. Different methods exist in order to achieve this: **modal synthesis**, **FDTD**, **digital waveguides**, *etc.*
- Faust has in its ***libraries*** ('`pm`' prefix) a good collection of ***physically-modeled instruments*** (*flutes*, *bells*, *guitars*, *voice*, *etc.*).

Faust: Advanced Synthesis (2/2)

- ***Physical model synthesis*** consists in ***emulating the physical properties of an instrument*** in order to reproduce its sound. Different methods exist in order to achieve this: **modal synthesis**, **FDTD**, **digital waveguides**, *etc.*
- Faust has in its ***libraries*** ('pm' prefix) a good collection of ***physically-modeled instruments*** (*flutes*, *bells*, *guitars*, *voice*, *etc.*).

- ***Granular synthesis*** is a method which uses overlapping '***grains***' (short fragments of sound, around **1 to 100 milliseconds**) from a ***buffer***, which are going to be played at *different speeds*, *phases*, *volume* and *frequency*.

Listing 48 `faust/examples/synthesis/granular.dsp`

```

```

Faust: Controls

Faust: Controls

- ***GUI elements*** in Faust can be ordered and gathered into horizontal/vertical or tab *layouts*, using '**groups**' or a special **path syntax** in their names.

Listing 49 `faust/examples/controls/groups.dsp`

```
import("stdfaust.lib");

// 'h:' horizontal group
// 'v:' vertical group
// 't:' tab group
// [0]: order of appearance
osc(n) = os.osci(
    hslider("h:oscillators/v:%n/[0]freq", 440, 20, 20000, 0.1)
) * hslider("h:oscillators/v:%n/[1]gain", 0, 0, 1, 0.01);

process = osc(0), osc(1);

```

Faust: Controls

- ***GUI elements*** in Faust can be ordered and gathered into horizontal/vertical or tab *layouts*, using '**groups**' or a special **path syntax** in their names.
- ***MIDI support*** can be added to any Faust program, simply by adding **metadata** on the **GUI elements** that we want to control.

Listing 50 `faust/examples/controls/midi.dsp`

```
import("stdfaust.lib");

// https://faustdoc.grame.fr/manual/midi/
declare options "[midi:on]";

// sliders
oscFreq = hslider("oscFreq [midi:ctrl 79]", 80, 50, 500, 0.01);
lfoFreq = hslider("lfoFreq [midi:ctrl 77]", 1, 0.01, 8, 0.01);
lfoRange = hslider("lfoRange [midi:ctrl 78]", 1000, 10, 5000, 0.01) : si.smoo;
noiseGain = hslider("noiseGain [midi:ctrl 80]", 0, 0, 1, 0.01) <: _*_;
masterVol = hslider("masterVol [midi:ctrl 81]", 0.8, 0, 1, 0.01) <: _*_;
panning = hslider("pan [midi:ctrl 49]", 0.5, 0, 1, 0.01)  : si.smoo;

// buttons
activateNoise = button("activateNoise [midi:ctrl 19]");
killSwitch = 1-button("killSwitch [midi:ctrl 20]");
LFO = os.lf_triangle(lfoFreq) * 0.5 + 0.5;

process = os.oscrc(440) * 0.25
        * killSwitch
        * os.sawtooth(oscFreq)
        + no.noise * noiseGain * activateNoise
        : fi.resonlp(LFO * lfoRange + 50, 5, 1)
        * masterVol
        <: _ * (1-panning), _ * panning;


```

Faust: Controls

- ***GUI elements*** in Faust can be ordered and gathered into horizontal/vertical or tab *layouts*, using '**groups**' or a special **path syntax** in their names.
- ***MIDI support*** can be added to any Faust program, simply by adding **metadata** on the **GUI elements** that we want to control.

- ***Open Sound Control*** (***OSC***) support follows the same *metadata* logic. ***OSC*** is a protocol for ***exchanging control data*** between *multimedia devices*, such as *synthesizers*, *DAWs*, *audio programming languages*, etc.

Listing 51 `faust/examples/controls/osc.dsp`

```
import("stdfaust.lib");

// https://faustdoc.grame.fr/manual/osc/
// listening port: 5510
declare options "[osc:on]";

// sliders
oscFreq = hslider("oscFreq [osc:/oscillator/freq]", 80, 50, 500, 0.01);
lfoFreq = hslider("lfoFreq [osc:/lfo/freq]", 1, 0.01, 8, 0.01);
lfoRange = hslider("lfoRange [osc:/lfo/range]", 1000, 10, 5000, 0.01) : si.smoo;
noiseGain = hslider("noiseGain [osc:/noise/gain]", 0, 0, 1, 0.01) <: _*_;
masterVol = hslider("masterVol [osc:/master/volume]", 0.8, 0, 1, 0.01) <: _*_;
panning = hslider("pan [osc:/master/panning]", 0.5, 0, 1, 0.01)  : si.smoo;

// buttons
activateNoise = button("activateNoise [osc:/noise/enabled]");
killSwitch = 1-button("killSwitch [osc:/master/kill]");
LFO = os.lf_triangle(lfoFreq) * 0.5 + 0.5;

process = os.oscrc(440) * 0.25
        * killSwitch
        * os.sawtooth(oscFreq)
        + no.noise * noiseGain * activateNoise
        : fi.resonlp(LFO * lfoRange + 50, 5, 1)
        * masterVol
        <: _ * (1-panning), _ * panning;


```

Faust: Controls

- ***GUI elements*** in Faust can be ordered and gathered into horizontal/vertical or tab *layouts*, using '**groups**' or a special **path syntax** in their names.
- ***MIDI support*** can be added to any Faust program, simply by adding **metadata** on the **GUI elements** that we want to control.
- ***Open Sound Control*** (***OSC***) support follows the same *metadata* logic. ***OSC*** is a protocol for ***exchanging control data*** between *multimedia devices*, such as *synthesizers*, *DAWs*, *audio programming languages*, etc.

- Finally ***HTTP control*** can be enabled in the same way, which allows *remote-control of a Faust program* within a **browser interface**.

Listing 52 `faust/examples/controls/http.dsp`

```
import("stdfaust.lib");

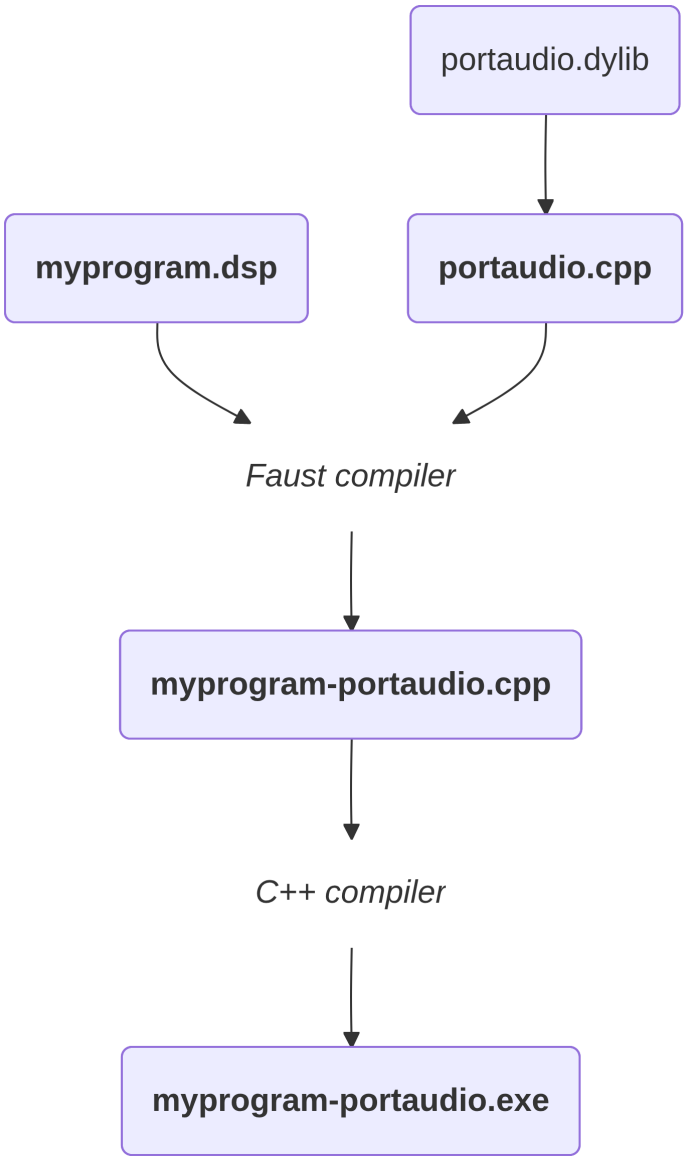
// https://faustdoc.grame.fr/manual/http/
// listening port: 5510
declare options "[http:on]";

```

Faust everywhere!

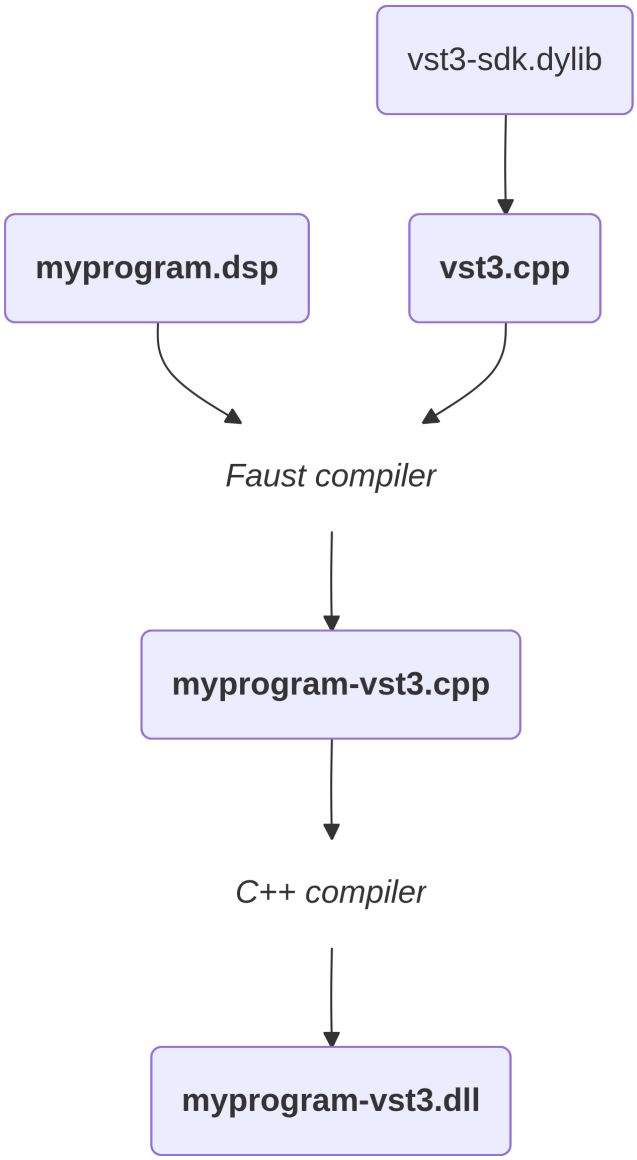
Faust: Exporting

- Faust manages to **export** an audio program to many different platforms & targets thanks to its ***architecture file system***.



Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single export target is materialized by a **specific architecture file**, written in C++.

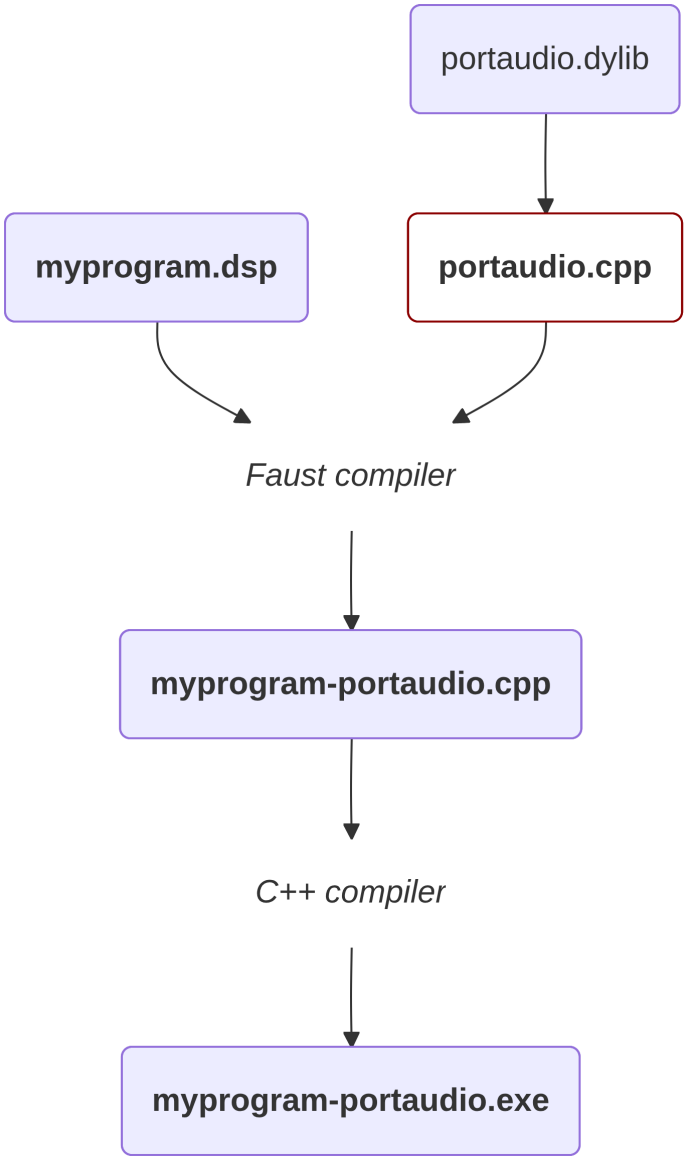


Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single **export target** is materialized by a **specific architecture file**, **written in C++**.

- The architecture file is **completely generic**, it doesn't change depending on the Faust

program.



Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single **export target** is materialized by a **specific architecture file**, **written in C++**.

- The architecture file is **completely generic**, it doesn't change depending on the Faust program.

Listing 53 `faust/architecture/pa-gtk.cpp`

```
// Include code from the faust and portaudio C++ libraries:
#include "faust/gui/FUI.h"
#include "faust/misc.h"
#include "faust/audio/portaudio-dsp.h"
#include "faust/gui/GTKUI.h"

// These are markers used to indicate to the Faust compiler
// where to 'paste' the generated DSP code
// from the .dsp file
<<includeIntrinsic>>
<<includeclass>>

// Create a DSP object
mydsp DSP;

// Program entrypoint:
int main(int argc, char *argv[])
{
  // Build the GTK User Interface:
  GUI* interface = new GTKUI (appname, &argc, &argv);
  DSP.buildUserInterface(interface);

  // Get the sample-rate from command-line arguments:
  long srate = (long)lopt(argv, "--sample-rate", 44100);
  // Get the buffer size from command-line arguments:
  int fpb = lopt(argv, "--buffer", 128);

  // Create the portaudio backend and initialize it:
  portaudio audio(srate, fpb);
  audio.init(appname, &DSP);

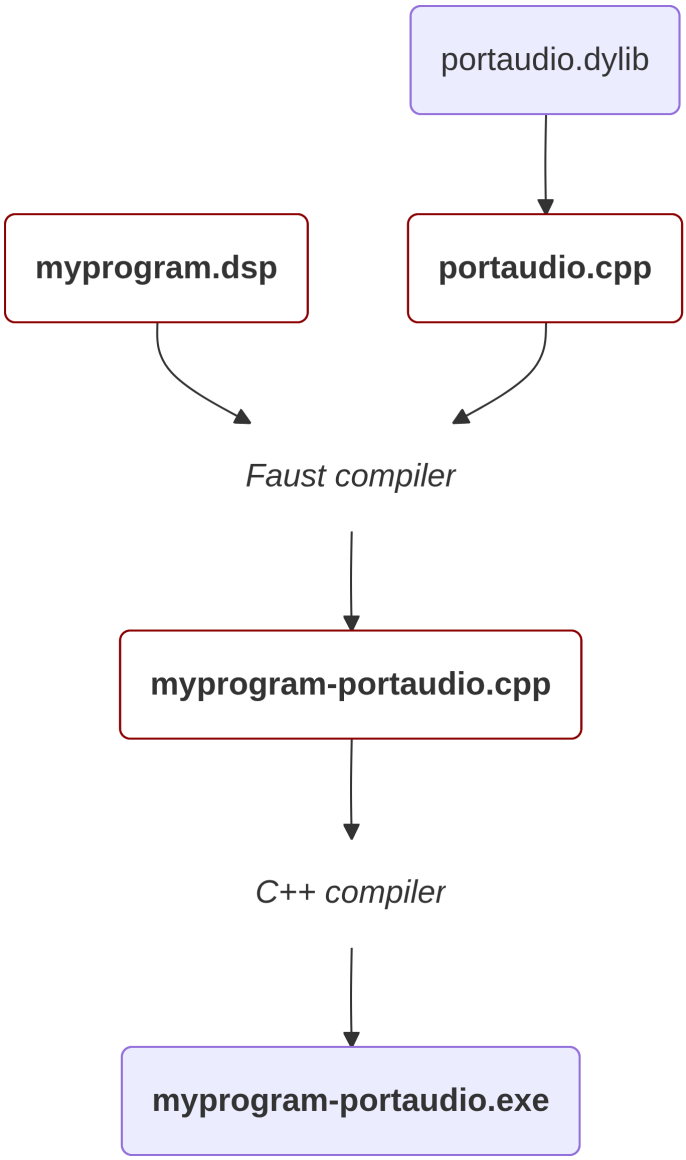
  // Run the interface:
  interface->run();

  // When the GTK Interface is closed, stop the audio
  // and deallocate the interface.
  audio.stop();
  delete interface;
}
```

Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single **export target** is materialized by a **specific architecture file**, **written in C++**.
- The architecture file is **completely generic**, it doesn't change depending on the Faust program.

- Instead, the **generated C++ code** for the **DSP program** is going to be inserted inside the architecture file, to create the **final compilable file**.



Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single **export target** is materialized by a **specific architecture file**, **written in C++**.
- The architecture file is **completely generic**, it doesn't change depending on the Faust program.

- Instead, the **generated C++ code** for the **DSP program** is going to be inserted inside the architecture file, to create the **final compilable file**.

Listing 54 `fm.cpp`

```
/* ------------------------------------------------------------
name: "fm"
Code generated with Faust 2.88.0 (https://faust.grame.fr)
Compilation options: -lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -
------------------------------------------------------------ */

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <math.h>

class mydsp : public dsp {
private:
  int fSampleRate;
  float fConst0;
  float fConst1;
  float fRec2[2];
  float fConst2;
  float fRec1[2];
  float fRec0[2];

public:
  mydsp() {}

  virtual int getNumInputs() {
    return 0;
  }
  virtual int getNumOutputs() {
    return 1;
  }

  virtual void instanceConstants(int sample_rate) {
    fSampleRate = sample_rate;
    fConst0 = std::min<float>(1.92e+05f, std::max<float>(1.0f, static_cast<float>(fSampleRate
    fConst1 = 4.4e+02f / fConst0;
    fConst2 = 1.0f / fConst0;
  }

  virtual void instanceClear() {
    for (int l0 = 0; l0 < 2; l0 = faust_wrap_add(l0, 1)) {
        fRec2[l0] = 0.0f;
    }
    for (int l1 = 0; l1 < 2; l1 = faust_wrap_add(l1, 1)) {
        fRec1[l1] = 0.0f;
    }
    for (int l2 = 0; l2 < 2; l2 = faust_wrap_add(l2, 1)) {
        fRec0[l2] = 0.0f;
    }
  }

  virtual void init(int sample_rate) {
```

Faust: Exporting

- Faust manages to **export** an audio program to many different platforms thanks to its ***architecture file system***.
- Every single **export target** is materialized by a **specific architecture file**, **written in C++**.
- The architecture file is **completely generic**, it doesn't change depending on the Faust program.

- Instead, the **generated C++ code** for the **DSP program** is going to be inserted inside the architecture file, to create the **final compilable file**.

Listing 55 `fm.cpp`

```
/* ------------------------------------------------------------
name: "fm"
Code generated with Faust 2.88.0 (https://faust.grame.fr)
Compilation options: -lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -
------------------------------------------------------------ */

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <math.h>

class mydsp : public dsp {
private:
  int fSampleRate;
  float fConst0;
  float fConst1;
  float fRec2[2];
  float fConst2;
  float fRec1[2];
  float fRec0[2];

public:
  mydsp() {}

  virtual int getNumInputs() {
    return 0;
  }
  virtual int getNumOutputs() {
    return 1;
  }

  virtual void instanceConstants(int sample_rate) {
    fSampleRate = sample_rate;
    fConst0 = std::min<float>(1.92e+05f, std::max<float>(1.0f, static_cast<float>(fSampleRate
    fConst1 = 4.4e+02f / fConst0;
    fConst2 = 1.0f / fConst0;
  }

  virtual void instanceClear() {
    for (int l0 = 0; l0 < 2; l0 = faust_wrap_add(l0, 1)) {
        fRec2[l0] = 0.0f;
    }
    for (int l1 = 0; l1 < 2; l1 = faust_wrap_add(l1, 1)) {
        fRec1[l1] = 0.0f;
    }
    for (int l2 = 0; l2 < 2; l2 = faust_wrap_add(l2, 1)) {
        fRec0[l2] = 0.0f;
    }
  }

  virtual void init(int sample_rate) {
```

Faust: Anatomy of a C++ audio program

- A **C/C++** `struct` represents a named data structure, in which one can **store data** of different types, and define **functions** associated (or not) with that data.

Listing 56 `sinewave.cpp`

```
struct sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init();

    // Processing function declaration:
    float process();
};
```

Faust: Anatomy of a C++ audio program

- A **C/C++** `struct` represents a named data structure, in which one can **store data** of different types, and define **functions** associated (or not) with that data (***methods***).
- Stored data can be **declared** with the syntax: `type mydata;`. It is called a **`struct` member.**

Listing 57 `sinewave.cpp`

```
struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init();

    // Processing function declaration:
    float process();
};
```

Faust: Anatomy of a C++ audio program

- A **C/C++** `struct` represents a named data structure, in which one can **store data** of different types, and define **functions** associated (or not) with that data.
- Stored data can be **declared** with the syntax: `type mydata`. They are called **`struct` members.**

- `struct` functions are declared with the syntax: `type myfunction()`. They are called ***methods***. The type represents **the value they return** after executing their instructions.

Listing 58 `sinewave.cpp`

```
struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init();

    // Processing function declaration:
    float process();
};
```

Faust: Anatomy of a C++ audio program

- A **C/C++** `struct` represents a named data structure, in which one can **store data** of different types, and define **functions** associated (or not) with that data.
- Stored data can be **declared** with the syntax: `type mydata`. They are called **`struct` members.**
- `struct` functions are declared with the syntax: `type myfunction()`. They are called ***methods***. The type represents **the value they return** after executing their instructions.

- We can **define what a method does** by opening a **curly-braces `{}` scope** after its name.

Listing 59 `sinewave.cpp`

```
struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init() {
        phase = 0;
    }

    // Processing function declaration:
    float process();
};
```

Faust: Anatomy of a C++ audio program

- A **C/C++** `struct` represents a named data structure, in which one can **store data** of different types, and define **functions** associated (or not) with that data.
- Stored data can be **declared** with the syntax: `type mydata`. They are called **`struct` members.**
- `struct` functions are declared with the syntax: `type myfunction()`. They are called ***methods***. The type represents **the value they return** after executing their instructions.
- We can **define what a method does** by opening a **curly-braces `{}` scope** after its name.

Listing 60 `sinewave.cpp`

```cpp
#include <cmath>

struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init() {
        phase = 0;
    }

    // Processing function declaration:
    float process() {
        // Increment phase sample-by-sample
        // for a fixed frequency of 440 Hz
        phase += 440.f / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        // Compute and return the sinewave value:
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};
```

Faust: Anatomy of a C++ audio program

- The `int main()` function is the ***entrypoint of a program***, it is the first function that is executed. It should **always return an integer value at the end**, either `0` if the program exited successfully, or otherwise an arbitrary **error code**.

Listing 61 `sinewave.cpp`

```cpp
#include <cmath>

struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init() {
        phase = 0;
    }

    // Processing function declaration:
    float process() {
        // Increment phase sample-by-sample
        // for a fixed frequency of 440 Hz
        phase += 440.f / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        // Compute and return the sinewave value:
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};

int main() {
    return 0;
}
```

Faust: Anatomy of a C++ audio program

- The `int main()` function is the ***entrypoint of a program***, it is the first function that is executed. It should **always return an integer value at the end**, either `0` if the program exited successfully, or otherwise an arbitrary **error code**.
- Inside of it, we can create an instance of our *sinewave* `struct`, store it into a **local variable**, call its *methods*, and visualize the result (**print**).

Listing 62 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init() {
        phase = 0;
    }

    // Processing function declaration:
    float process() {
        // Increment phase sample-by-sample
        // for a fixed frequency of 440 Hz
        phase += 440.f / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        // Compute and return the sinewave value:
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};

int main() {
    // Create an instance of our custom sinewave type,
    // store it into a local variable:
    Sinewave sine;

    // Call its 'init' function:
    sine.init();

    // Get a sample from it, store it in a local variable:
    float sample = sine.process();

    // Print it:
    printf("Sample value: %f", sample);
    return 0;
}
```

Faust: Anatomy of a C++ audio program

- The `int main()` function is the ***entrypoint of a program***, it is the first function that is executed. It should **always return an integer value at the end**, either `0` if the program exited successfully, or otherwise an arbitrary **error code**.
- Inside of it, we can create an instance of our *sinewave* `struct`, store it into a **local variable**, call its *methods*, and visualize the result (**print**).

- We can now **repeat** the process as many times as we want by encapsulating the `process()`and `print()` function calls in a `for` **loop**.

Listing 63 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Sinewave {
    // Stored data: 32-bits floating-point value:
    float phase;

    // Initialization function declaration:
    void init() {
        phase = 0;
    }

    // Processing function declaration:
    float process() {
        // Increment phase sample-by-sample
        // for a fixed frequency of 440 Hz
        phase += 440.f / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        // Compute and return the sinewave value:
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};

int main() {
    // Create an instance of our custom sinewave type,
    // store it into a local variable:
    Sinewave sine;

    // Call its 'init' function:
    sine.init();

    // Now, the following is going to be repeated 100 times:
    for (int i = 0; i < 100; i++) {
        float sample = sine.process();
        printf("Sample value: %f\n", sample);
    }
    return 0;
}
```

Faust: Anatomy of a C++ audio program

- We could also have made a ***separate structure*** for the '*phase*' computation, to make it **reusable** in other components or **waveform oscillators**.

Listing 64 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Phasor {
    float phase;

    void init() {
        phase = 0;
    }

    float process() {
        // Increment phase sample-by-sample
        // for a fixed frequency of 440 Hz
        phase += 440.f / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        return phase;
    }
};

struct Sinewave {
    // Stored data: an instance of the Phasor struct:
    Phasor phasor;

    // Initialization function declaration:
    void init() {
        phasor.init();
    }

    // Processing function declaration:
    float process() {
        // Compute and return the sinewave value:
        float phase = phasor.process();
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};

int main() {
    // Create an instance of our custom sinewave type,
    // store it into a local variable:
    Sinewave sine;

    // Call its 'init' function:
    sine.init();

    // Now, the follow"ing is going to be repeated 100 times:
    for (int i = 0; i < 100; i++) {
```

Faust: Anatomy of a C++ audio program

- We could also have made a ***separate structure*** for the '*phase*' computation, to make it **reusable** in other components or **waveform oscillators**.
- Additionally, let's make the **frequency** of the oscillator **variable**, instead of fixed to `440`.

Listing 65 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Phasor {
    float phase;

    // We add another struct member:
    float frequency;

    // Add a float argument to the init function:
    void init(float freq) {
        phase = 0;
        frequency = freq;
    }

    float process() {
        // Increment phase sample-by-sample
        // given the frequency set at initialization:
        phase += frequency / 48000.f;
        // When phase reaches its maximum (1.0),
        // start from 0.0 again:
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        return phase;
    }
};

struct Sinewave {
    // Stored data: 32-bits floating-point value:
    Phasor phasor;

    // Initialization function declaration:
    void init(float freq) {
        phasor.init(freq);
    }

    // Processing function declaration:
    float process() {
        // Compute and return the sinewave value:
        float phase = phasor.process();
        float sine = sin(phase * 2.f * M_PI);
        return sine;
    }
};

int main() {
    // Create an instance of our custom sinewave type,
    // store it into a local variable:
    Sinewave sine;

```

Faust: Anatomy of a C++ audio program

- We could also have made a ***separate structure*** for the '*phase*' computation, to make it **reusable** in other components or **waveform oscillators**.
- Additionally, let's make the **frequency** of the oscillator **variable**, instead of fixed to `440`.

- Now let's make the same `struct` for *triangle*, *sawtooth*, and *square* waves, and let's also add an **amplitude parameter** to them.

Listing 66 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Phasor {
    float phase;
    float frequency;

    void init(float freq) {
        phase = 0;
        frequency = freq;
    }

    float process() {
        phase += frequency/48000.f;
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        return phase;
    }
};

struct Sinewave {
    Phasor phasor;

    void init(float freq) {
        phasor.init(freq);
    }

    float process() {
        float phase = phasor.process();
        float sine = sin(phase * M_PI * 2.f);
        return sine;
    }
};

struct Square {
    Phasor phasor;

    void init(float freq) {
        phasor.init(freq);
    }

    float process() {
        float phase = phasor.process();
        float square = phase < 0.5f ? -1.f : 1.f;
        return square;
    }
};

struct Sawtooth {
    Phasor phasor;
```

Faust: Anatomy of a C++ audio program

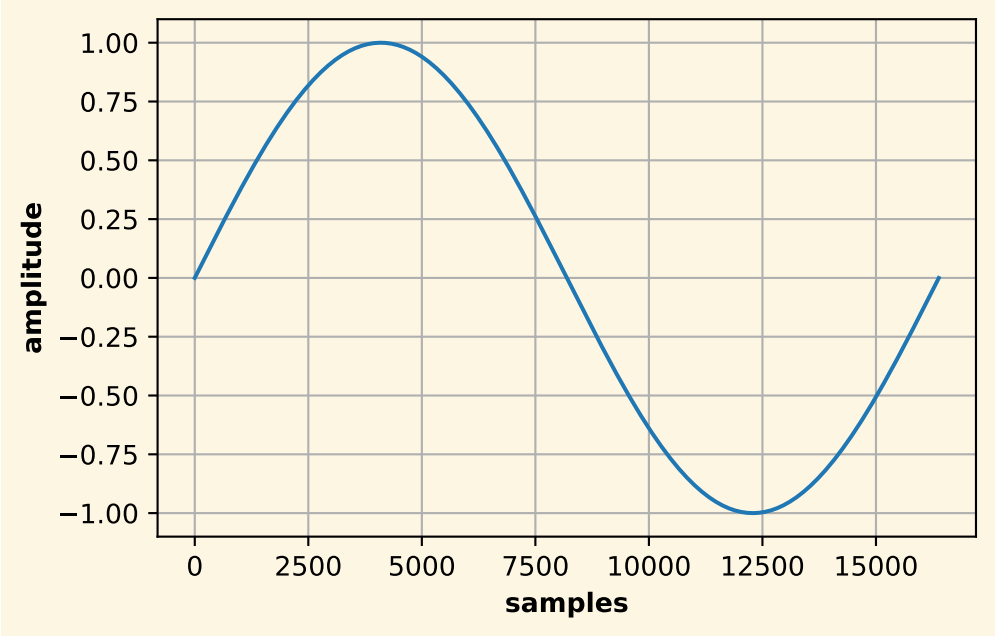
- We could also have made a ***separate structure*** for the '*phase*' computation, to make it **reusable** in other components or **waveform oscillators**.
- Additionally, let's make the **frequency** of the oscillator **variable**, instead of fixed to `440`.
- Now let's make the same `struct` for *triangle*, *sawtooth*, and *square* waves, and let's also add an **amplitude parameter** to them.

- In order to make the computation a bit lighter, we could also store the waveforms in **pre-computed table/arrays**.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(16384)
y = np.sin(x/16384 * np.pi * 2);

plt.figure(facecolor="#fdf6e3")
ax = plt.gca()
ax.set_facecolor("#fdf6e3")
plt.grid()
plt.plot(x, y)
plt.xlabel("samples", fontweight='bold')
plt.ylabel('amplitude', fontweight='bold')
plt.show()
```



Faust: Anatomy of a C++ audio program

- We could also have made a ***separate structure*** for the '*phase*' computation, to make it **reusable** in other components or **waveform oscillators**.
- Additionally, let's make the **frequency** of the oscillator **variable**, instead of fixed to `440`.
- Now let's make the same `struct` for *triangle*, *sawtooth*, and *square* waves, and let's also add an **amplitude parameter** to them.

- In order to make the computations a bit lighter, we could also store the waveforms in **pre-computed table/arrays**.

Listing 67 `sinewave.cpp`

```cpp
#include <cmath>
#include <cstdio>

struct Phasor {
    float phase;
    float frequency;

    void init(float freq) {
        phase = 0;
        frequency = freq;
    }

    float process() {
        phase += frequency/48000.f;
        if (phase >= 1.f) {
            phase -= 1.f;
        }
        return phase;
    }
};

const int TABLE_SIZE = 16384;

struct Sinewave {
    Phasor phasor;
    float table[TABLE_SIZE];

    void init(float freq) {
        phasor.init(freq);
        for (int x = 0; x < TABLE_SIZE; x++) {
             float fx = float(x);
             float phase = fx / TABLE_SIZE;
             float y = sin(phase * M_PI * 2.f);
             table[x] = y;
        }
    }
    float process() {
        float phase = phasor.process();
        float sine = table[int(phase * TABLE_SIZE)];
        return sine;
    }
};

int main() {
    Sinewave sine;
    sine.init(440.f);

    for (int i = 0; i < 48000; i++) {
        float sample = sine.process();
        int n = sample * 40 + 40;
        for (int s = 0; s < 100; ++s) {
```

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.
- A plugin consists in **multiple files**, set in a specific **directory structure**.

Listing 68 `vcv-rack/plugins/`

```
-- my_plugin/
  -- Makefile                | Build
  -- plugin.json             | Plugin Metadata
  -- src/
      -- plugin.cpp          | Plugin interface
      -- plugin.hpp
      -- my_plugin.cpp       | Actual plugin code
  -- res/
      -- my_plugin.svg       | Plugin appearance
```

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.
- A plugin consists in **multiple files**, set in a specific **directory structure**.
- The **`plugin.json`** file contains the plugin's **metadata**: its name, author, version, etc.

Listing 69 `my_plugin/plugin.json`

```
{
  "slug": "emeraude",
  "name": "Emeraude",
  "version": "2.2.0",
  "license": "MIT",
  "brand": "Emeraude",
  "author": "Pierre Cochard",
  "authorEmail": "pierre.cochard@insa-lyon.fr",
  "authorUrl": "",
  "pluginUrl": "",
  "manualUrl": "",
  "sourceUrl": "",
  "donateUrl": "",
  "changelogUrl": "",
  "modules": [
    {
      "slug": "my_module",
      "name": "My Module",
      "description": "",
      "tags": ["Oscillator"]
    }
  ]
}
```

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.
- A plugin consists in **multiple files**, set in a specific **directory structure**.
- The `plugin.json` file contains the plugin's **metadata**: its name, author, version, etc.
- The `plugin.hpp` & `plugin.cpp` files declare the plugin instance data (nothing to do in these files for now).

Listing 70 `my_plugin/src/plugin.hpp`

```hpp
#pragma once
#include <rack.hpp>

using namespace rack;

// Declare the Plugin, defined in plugin.cpp
extern Plugin* pluginInstance;

// Declare each Model, defined in each module source file
extern Model* modelMyModule;
```

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.
- A plugin consists in **multiple files**, set in a specific **directory structure**.
- The `plugin.json` file contains the plugin's **metadata**: its name, author, version, etc.
- The **`plugin.hpp`** & **`plugin.cpp`** files declare the plugin instance data (nothing to do in these files for now).

Listing 71 `my_plugin/src/plugin.cpp`

```cpp
#include "plugin.hpp"

Plugin* pluginInstance;

void init(Plugin* p) {
    pluginInstance = p;

    // Add modules here
    p->addModel(modelMyModule);
}
```

Creating a simple VCV-Rack plugin

- **VCV-Rack** has a simple but powerful **plugin system**, which allows user to program their own **third-party modules**.

- A plugin consists in **multiple files**, set in a specific **directory structure**.
- The `plugin.json` file contains the plugin's **metadata**: its name, author, version, etc.
- The `plugin.hpp` & `plugin.cpp` files declare the plugin instance data (nothing to do in these files for now).

- Finally, **`myplugin.cpp`**, must be structured in the following way:

Listing 72 `my_plugin/src/myplugin.cpp`

```

#include "plugin.hpp"

// It must inherit from Module
struct MyModule : Module {
    // Declare parameter ids:
    enum ParamId {
        // Last value is equal to the number of items in the enum:
        FREQUENCY_PARAM = 0,
        PARAMS_LEN = 1
    };
    // Declare input ids:
    enum InputId {
        INPUTS_LEN
    };
    // Declare output ids:
    enum OutputId {
        MAIN_OUTPUT,
        OUTPUTS_LEN
    };
    // Declare light (LED) ids:
    enum LightId {
        LIGHTS_LEN
    };

    // Initialization function (constructor):
    MyModule() {
        config(PARAMS_LEN, INPUTS_LEN, OUTPUTS_LEN, LIGHTS_LEN);
        // Register parameters:
        // min, max, default, name, unit
        configParam(FREQUENCY_PARAM, 50.f, 100.f, 75.f, "frequency", "Hz");

        // Register outputs:
        configOutput(MAIN_OUTPUT, "main");
    }

    void process(ProcessArgs const& args) override {
        // Processing goes here:
    }
};

// The 'widget' part, meaning its graphical representation:
struct MyModuleWidget : ModuleWidget {
    MyModuleWidget(MyModule* module) {
        // Link the widget with the module:
        setModule(module);

        // Load the .svg panel in res/ directory:
        setPanel(createPanel(asset::plugin(pluginInstance, "res/my_module.svg")));

        // Create and place decorative screws:
```